

Database dan Authentication

SQLite · Prisma ORM · bcrypt · JWT · Middleware

Agenda

01

Database & SQLite

- Konsep relasional
- Setup Prisma ORM
- Schema & migration

02

Prisma CRUD

- Create, Read, Update, Delete
- Filter & pagination
- Error handling

03

Authentication

- Hash password (bcrypt)
- Token stateless (JWT)
- Middleware authGuard

04

Praktik

- Produk API + SQLite
- Register & Login
- Protected routes

01

Database & SQLite

Konsep relasional, setup Prisma ORM, schema & migration

Apa itu Database Relasional?

Analogi: Lemari Arsip

- Setiap laci = satu tabel (users, products)
- Setiap map = satu baris data
- Kolom form = field (nama, harga, stok)

Kenapa "Relasional"?

- Tabel bisa saling terhubung lewat foreign key.
- Contoh: products menyimpan userId.

Istilah penting

- Primary Key — ID unik setiap baris
- Foreign Key — referensi ke tabel lain
- Migration — script untuk buat/ubah tabel
- ORM — query DB pakai JavaScript

-- Tabel: users

id	email	role
1	budi@...	user
2	siti@...	admin

↑ primary key
↓ foreign key

-- Tabel: products

id	nama	harga	userId
1	Apel	15000	1
2	Mangga	25000	1
3	Jeruk	12000	2

userId = 1 → milik budi@...
userId = 2 → milik siti@...

Setup SQLite + Prisma ORM

1 Install packages

```
npm install prisma @prisma/client
```

2 Init Prisma

```
npx prisma init --datasource-provider sqlite
```

3 Atur DATABASE_URL

```
DATABASE_URL="file:./dev.db"
```

4 Tulis schema.prisma

```
Definisikan model User & Product
```

5 Jalankan migration

```
npx prisma migrate dev --name init
```

6 Buat db.js (singleton)

```
const prisma = new PrismaClient()
```

```
# 1. Install
npm install prisma @prisma/client

# 2. Init dengan SQLite
npx prisma init \
  --datasource-provider sqlite

# .env
DATABASE_URL="file:./dev.db"
JWT_SECRET="kunci_rahasia_panjang"

# 3. Migrate
npx prisma migrate dev --name init

# 4. Lihat data di browser
npx prisma studio
# buka http://localhost:5555

# db.js
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
module.exports = prisma;
```

Schema Prisma — Mendefinisikan Tabel

```
// prisma/schema.prisma

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id          Int      @id @default(autoincrement())
  email       String   @unique
  password    String
  role        String   @default("user")
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  products    Product[]
}

model Product {
  id          Int      @id @default(autoincrement())
  nama        String
  harga       Int
  stok        Int      @default(0)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  userId      Int?
  user        User?    @relation(
    fields: [userId], references: [id])
}
```

Anotasi Prisma

- `@id` — primary key tabel
- `@default(autoincrement())` — otomatis 1, 2, 3...
- `@unique` — nilai tidak boleh duplikat
- `@default("user")` — nilai default jika kosong
- `@default(now())` — otomatis waktu sekarang
- `@updatedAt` — otomatis update saat edit

Tanda tanya (?) = nullable

- `userId Int?` → boleh null
- `String tanpa ?` → wajib diisi

Product[] bukan kolom di DB

- Virtual field untuk relasi di kode.
- Prisma tidak buat kolom ini di tabel.

Migration — Dari Schema ke Database

Apa yang terjadi saat migrate?

- Prisma baca schema.prisma
- Bandingkan dengan kondisi database sekarang
- Buat file SQL di prisma/migrations/
- Jalankan SQL ke file dev.db

Perintah

- migrate dev --name init → pertama kali
- migrate dev --name xxx → setelah ubah schema
- migrate status → cek sudah dijalankan belum

Yang harus di-commit ke Git

- prisma/migrations/ → wajib commit
- dev.db → jangan commit (.gitignore)

```
-- SQL yang dibuat OTOMATIS oleh Prisma
-- (tidak perlu tulis manual)

CREATE TABLE "User" (
  "id"          INTEGER NOT NULL
               PRIMARY KEY AUTOINCREMENT,
  "email"       TEXT NOT NULL,
  "password"    TEXT NOT NULL,
  "role"        TEXT NOT NULL DEFAULT 'user',
  "createdAt"   DATETIME NOT NULL
               DEFAULT CURRENT_TIMESTAMP,
  "updatedAt"   DATETIME NOT NULL
);

CREATE UNIQUE INDEX "User_email_key"
ON "User"("email");

CREATE TABLE "Product" (
  "id"          INTEGER NOT NULL PRIMARY KEY,
  "nama"        TEXT NOT NULL,
  "harga"       INTEGER NOT NULL,
  "stok"        INTEGER NOT NULL DEFAULT 0,
  "userId"      INTEGER,
  FOREIGN KEY ("userId")
    REFERENCES "User"("id")
);
```

02

Prisma CRUD

Create, Read, Update, Delete — query database dengan JavaScript

Prisma CRUD — Create & Read

Create — tambah data baru

```
const p = await prisma.product.create({
  data: {
    nama: 'Apel Fuji',
    harga: 15000,
    stok: 100,
  },
});

// hasil:
// { id: 1, nama: "Apel Fuji",
//   harga: 15000, stok: 100 }

// Kolom @default otomatis terisi.
```

Read — ambil data dari database

```
// ambil semua produk
const all = await prisma.product.findMany({
  orderBy: { createdAt: 'desc' },
  take: 10, // LIMIT 10
  skip: 0, // mulai dari baris ke-0
});

// ambil satu berdasarkan id
const p = await prisma.product.findUnique({
  where: { id: +req.params.id }
});
if (!p) return res.status(404)
.json({ error: 'Tidak ditemukan' });

// cari dengan kondisi
const hasil = await prisma.product.findMany({
  where: {
    nama: { contains: 'apel' },
    harga: { gte: 5000 },
  },
});
```

Prisma CRUD — Update, Delete & Pagination

Update & Delete

```
// UPDATE
const updated = await prisma.product.update({
  where: { id: +req.params.id },
  data: { harga: 20000, stok: 80 },
});

// DELETE
await prisma.product.delete({
  where: { id: +req.params.id }
});
res.status(204).send();

// Jika id tidak ada → error P2025
} catch (e) {
  if (e.code === "P2025")
    return res.status(404)
      .json({ error: 'Tidak ada' });
  next(e);
}
```

Pagination — data per halaman

```
// GET /api/products?page=2&limit=10
const page = +(req.query.page || 1);
const limit = +(req.query.limit || 10);

const [data, total] = await Promise.all([
  prisma.product.findMany({
    take: limit,
    skip: (page - 1) * limit,
    orderBy: { createdAt: 'desc' },
  }),
  prisma.product.count(),
]);

res.json({
  data,
  meta: {
    total, page,
    totalPages: Math.ceil(total / limit),
  }
});

// Promise.all = dua query bersamaan
```

Relasi (JOIN), Aggregate & Error Codes

include — setara JOIN di SQL

- Ambil data dari tabel lain sekaligus
- Gunakan select di dalam include untuk memilih kolom yang mau diambil.

Aggregate — statistik data

- count() → hitung jumlah baris
- aggregate() → avg, min, max, sum

Error codes Prisma

- P2002 — unique constraint gagal (email sudah terdaftar)
- P2025 — record tidak ditemukan (saat update / delete)
- Doc Reference : <https://www.prisma.io/docs/orm/reference/error-reference>

```
// include = JOIN
const list = await prisma.product.findMany({
  include: {
    user: {
      select: { email: true, role: true }
    }
  }
});
// hasil: [..., user: { email, role }]

// COUNT & AGGREGATE
const total = await prisma.product.count();
const stats = await prisma.product.aggregate({
  _avg: { harga: true },
  _min: { harga: true },
  _max: { harga: true },
});

// ERROR HANDLING
} catch (e) {
  if (e.code === 'P2002')
    return res.status(400).json({error: 'Duplikat'});
  if (e.code === 'P2025')
    return res.status(404).json({error: 'Tidak ada'});
  next(e);
}
```

03

Authentication

Hash password dengan bcrypt, token stateless dengan JWT

Authentication vs Authorization

Authentication (AuthN)

"Kamu siapa?"

- Verifikasi identitas pengguna.
- Seperti: tunjukkan KTP sebelum masuk.

Di API kita

- POST /api/auth/login
- Kirim email + password
- Jika cocok → dapat JWT token
- Token = kartu tanda pengenal digital

Status HTTP

- 401 Unauthorized
Belum login atau token tidak valid

Authorization (AuthZ)

"Kamu boleh ngapain?"

- Verifikasi hak akses pengguna.
- Seperti: tamu hotel tidak boleh ke dapur.

Di API kita

- User sudah login, tapi ada batasan.
- User biasa tidak bisa hapus user lain.
- Hanya admin yang bisa /api/admin.

Status HTTP

- 403 Forbidden
Sudah login, tapi tidak punya izin

bcrypt — Hash Password

Kenapa tidak simpan teks biasa?

- Kalau database bocor, semua password terbaca.

Solusi: Hash

- Mengubah teks menjadi string acak.
- "rahasia123" → "\$2a\$10\$N9qo8u..."
- Tidak bisa dikembalikan ke aslinya.

Cost factor (angka 10)

- $2^{10} = 1024$ iterasi — sengaja lambat.
- Makin tinggi = makin susah brute force.
- Nilai 10 cukup untuk kebanyakan app.

Gunakan bcryptjs

- Pure JavaScript, tidak perlu kompilasi.
- npm install bcryptjs

```
const bcrypt = require('bcryptjs');

// REGISTER — hash dulu sebelum simpan
const hash = await bcrypt.hash(
  'rahasia123', // password dari form
  10           // cost factor
);
// "$2a$10$N9qo8u..." — selalu berbeda

// Simpan HASH, bukan plaintext!
await prisma.user.create({
  data: { email, password: hash }
});

// LOGIN — verifikasi
const cocok = await bcrypt.compare(
  'rahasia123', // dari form login
  hash         // dari database
);

// true → password benar ✓
// false → password salah ✗
```

JWT — Token Stateless

Apa itu JWT?

- Tanda pengenalan digital setelah login.
- Server tidak perlu simpan session.

Struktur (3 bagian dipisah titik)

- Header — algoritma (HS256)
- Payload — data: userId, email, role, exp
- Signature — tanda tangan dengan JWT_SECRET

Penting: Payload bisa dibaca siapa saja

- Payload hanya base64, bukan enkripsi.
- Jangan simpan password di JWT.

expiresIn

- "7d" = 7 hari, "1h" = 1 jam
- Token expired → user harus login ulang.

```
const jwt = require('jsonwebtoken');

// Buat token setelah login berhasil
const token = jwt.sign(
  {
    userId: user.id,
    email: user.email,
    role: user.role,
  },
  process.env.JWT_SECRET,
  { expiresIn: '7d' }
);

// Verifikasi token di middleware
try {
  const decoded = jwt.verify(
    token,
    process.env.JWT_SECRET
  );
  // { userId, email, role, iat, exp }
  req.user = decoded;
  next();
} catch (err) {
  res.status(401).json({ error: 'Token tidak valid' });
}
```

Endpoint Register

1 Validasi input

Email valid? Password min 8 karakter?



2 Cek duplikat email

findUnique, tolak jika sudah ada



3 Hash password

bcrypt.hash(password, 10)



4 Simpan ke database

prisma.user.create(...)



5 Balas 201 Created

id + email, JANGAN kirim hash



```
router.post('/register', async (req, res, next) => {
  try {
    const { email, password } = req.body;

    // 1. Validasi
    if (!email || !password)
      return res.status(400).json({ error: 'Wajib isi' });
    if (password.length < 8)
      return res.status(400)
        .json({ error: 'Password min 8 karakter' });

    // 2. Cek duplikat
    const ada = await prisma.user.findUnique(
      { where: { email } });
    if (ada) return res.status(400)
      .json({ error: 'Email sudah terdaftar' });

    // 3 & 4. Hash & Simpan
    const hash = await bcrypt.hash(password, 10);
    const user = await prisma.user.create({
      data: { email, password: hash },
      select: { id: true, email: true }
    });

    // 5. Balas
    res.status(201).json({ message: 'Berhasil', user });
  } catch (e) { next(e); }
});
```


Endpoint Login

1 Cari user berdasarkan email

findUnique({ where: { email } })



2 Verifikasi password

bcrypt.compare(password, user.password)



3 Buat JWT token

jwt.sign({ userId, email, role }, secret)



4 Kirim token ke client

Client simpan & sertakan di request berikutnya



Pesan error sengaja sama untuk email & password, agar attacker tidak tahu mana yang salah.

```
router.post('/login', async (req, res, next) => {
  try {
    const { email, password } = req.body;

    // 1. Cari user
    const user = await prisma.user.findUnique(
      { where: { email } });

    // 2. Verifikasi
    const ok = user &&
      await bcrypt.compare(password, user.password);
    if (!ok) return res.status(401)
      .json({ error: 'Email atau password salah' });

    // 3. Buat token
    const token = jwt.sign(
      { userId: user.id, email: user.email,
        role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '7d' }
    );

    // 4. Kirim
    res.json({
      token,
      user: { id: user.id, email: user.email }
    });
  } catch (e) { next(e); }
});
```

Middleware — authGuard & adminOnly

Apa itu Middleware?

- Fungsi yang berjalan sebelum route handler.
- Seperti satpam di pintu — cek dulu.

authGuard

- Baca header: Authorization: Bearer <token>
- Panggil jwt.verify() untuk validasi.
- Valid → isi req.user, lanjut (next()).
- Tidak valid → stop, balas 401.

adminOnly

- Pasang setelah authGuard.
- Cek req.user.role === "admin".
- Bukan admin → balas 403 Forbidden.

Cara pasang di index.js

- app.use('/api/products', authGuard, router)

```
// middleware/authGuard.js
const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
  const auth = req.headers.authorization;
  if (!auth || !auth.startsWith('Bearer '))
    return res.status(401).json({ error: 'Login dulu' });

  const token = auth.slice(7);
  try {
    req.user = jwt.verify(
      token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Token tidak valid' });
  }
};

// middleware/adminOnly.js
module.exports = (req, res, next) => {
  if (req.user?.role !== 'admin')
    return res.status(403).json({ error: 'Admin only' });
  next();
};

// index.js
app.use('/api/products', authGuard,
  require('./routes/products'));
```

Ringkasan Materi

Database & SQLite

SQLite · Prisma ORM · schema.prisma · migration · Prisma Studio

Prisma CRUD

create · findMany · findUnique · update · delete · filter · include

Authentication

bcrypt.hash() · bcrypt.compare() · jwt.sign() · jwt.verify()

Protected Routes

authGuard middleware · adminOnly middleware · 401 vs 403

Selanjutnya: Praktik — terapkan semua materi di atas